

Choir Geek

Project Documentation

From Prototype to Product

Version 8.2 • 26 March 2026

A personal practice tool for choir singers, built to evolve into a scalable product

1. Project Origin & Concept

Choir Geek began as a personal Home Assistant dashboard panel, created to solve a specific problem faced by contemporary rock choir members: the need to practise at home using provided MP3 harmony tracks alongside PDF lyric sheets, with no purpose-built tool available to manage both in one place.

The creator is a Rock Choir member who also leads a smaller choir group, and recognised that a simple, accessible tool for this workflow was missing from the market. What started as a Home Assistant Lovelace panel rapidly evolved into a fully-featured standalone practice tool with real commercial potential.

The Core Problem

- Choir members are provided with multiple MP3 files per song (one per harmony part) and a single PDF lyric sheet
- No existing tool combines MP3 playback with PDF viewing in a single, simple interface
- The target demographic (middle-aged and older adults) requires a non-technical, intuitive experience
- Existing solutions are either too complex, platform-locked, or designed for a different use case

2. Technical Build

2.1 File Structure

The application is a self-contained HTML file served via a standalone Python HTTP server. The Home Assistant dependency has been removed — the app now runs on any machine with Python installed. All files live in a single choir folder:

File / Folder	Purpose
index.html	The Thin HTML shell (≈447 lines) linking 4 CSS modules and 3 JS modules. Served by Caddy at https://choirgeek.duckdns.org
generate_tracks.py	Python script that scans the choir folder and generates tracks.json
tracks.json	Auto-generated manifest consumed by the app on page load
Choir_Files_Renamer.py	Utility script to bulk-rename MP3 files to the required naming convention
[Song Name]/	Individual folders per song, each containing MP3s and one PDF

The Home Assistant integration (start_server.sh, shell commands, Lovelace card) is no longer required. See Section 4 if you wish to continue using HA as a front-end.

2.2 Folder & Naming Convention

Each song lives in its own subfolder. The naming convention must be consistent:

A Little Respect/ A Little Respect - Bass.mp3 A Little Respect - Full Harmony.mp3 A Little Respect - Male Bass.mp3 A Little Respect - Lyrics.pdf

The generate_tracks.py script extracts the part label by stripping the "Song Name - " prefix and .mp3 extension. There is always exactly one PDF per folder.

2.3 Supported Vocal Parts

The generate_tracks.py and Choir_Files_Renamer.py scripts recognise eleven distinct part labels, matched in priority order:

Part Label	Notes
Full Harmony	Defaults to this part when a song is selected
Soprano	
Lower Soprano	
Upper Alto	
Alto	
Lower Alto	
Bass	
Upper Bass	
Lower Bass	
Male Bass	
Upper Male Bass	Matched before Male Bass to avoid partial match

Parts are sorted in the dropdown in vocal range order (soprano to bass), not alphabetically.

2.4 The Track Manifest (tracks.json)

```
[ { "song": "A Little Respect",    "pdf": "A Little Respect/A Little Respect - Lyrics.pdf",    "parts": [      { "label": "Full Harmony", "src": "A Little Respect/..." },      { "label": "Bass", "src": "..." } ] ] ]
```

2.5 Running the App

The app is served by Caddy running as a system service on a Mac Mini (192.168.1.107). Files are stored on an external NVMe drive at /Volumes/1TB-NVME/Choir/. The main application file is index.html.

Note (v6.0+): The app uses native ES modules and must be served over HTTP — opening index.html directly as a file:// URL will not work. The Caddy server handles this automatically; for local testing use: python3 -m http.server in the Choir folder.

Important: as of v6.0 the app uses native ES modules and must be served over HTTP. Opening index.html directly as a file:// URL will not work — use the Caddy server or any local HTTP server (e.g. `python3 -m http.server`).

Public URL (HTTPS):

`https://choirgeek.duckdns.org`

SSL is handled automatically by Caddy via Let's Encrypt. The certificate auto-renews — no manual intervention required.

Caddyfile (/opt/homebrew/etc/Caddyfile):

```
choirgeek.duckdns.org
root * /Volumes/1TB-NVME/Choir
file_server
```

Infrastructure notes:

- DuckDNS keeps choirgeek.duckdns.org pointed at the home external IP. The updater runs on the Plex server (also 24/7) — both machines share the same external IP so no separate updater is needed on the Mac Mini
- Router port forwarding: external port 80 → 192.168.1.107:80 and external port 443 → 192.168.1.107:443
- Mac Mini local IP is manually assigned (static): 192.168.1.107
- Caddy must be granted Full Disk Access in macOS System Settings → Privacy & Security → Full Disk Access (/opt/homebrew/bin/caddy) to read files from the external NVMe drive after a reboot
- Caddy runs as a root system service (`sudo brew services start caddy`) so it can bind to ports 80 and 443

2.6 PDF Rendering

The PDF viewer uses Mozilla's PDF.js library to render PDF pages to HTML canvas elements. PDF.js is loaded from the Cloudflare CDN and requires internet access on first load. If the server is air-gapped, pdf.min.js and pdf.worker.min.js can be self-hosted in the choir folder.

3. Complete Feature Set

3.1 Song & Part Selection

- Two-stage selector: Song then Part
- Song list loaded dynamically from tracks.json on page load
- Selecting a song immediately loads the PDF and populates the Part selector
- Part defaults to "Full Harmony" if present, otherwise first in vocal range order — unless the user has set a personal default. Tap the ★ star button next to the Part selector to save the current part as your global default; every song will then open straight to that part. Tap the star again to clear it
- **Loop persists on part switch:** switching harmony part within the same song preserves phrase loop points — drill a passage in Full Harmony then switch to your specific part without losing the loop. Only selecting a new song clears it
- Changing song clears any active phrase loop and resets position

- Refresh button reloads tracks.json without a page reload

3.2 Audio Player

- Play/pause, restart (⏮ Start), skip back 10s, skip forward 10s
- Skip buttons are always gold to indicate they are always interactive
- Scrubbing progress bar with current time and total duration, purple-to-gold gradient fill
- Animated gradient accent line along the top of the player bar during playback
- Volume slider — always active; initialises from saved setting
- Mute button — animates slider smoothly to 0 and back to previous level
- **Slow Down Playback:** First tap drops to -10% (0.90×); each - step takes it further: -18%, -25%, -30%, -35% (0.65× minimum). Pitch stays identical at every speed — powered by a WSOLA pitch-preserving engine (SoundTouch algorithm). Use + to ease back up, or × to snap straight to 1×. Speed resets to 1× on every new song
- Autoplay toggle — session-only, always resets to off on launch. Accessible via the settings panel (gear icon) in v7.9+

3.3 Phrase Looper

The A/B loop has been replaced with a more intuitive Phrase Looper designed for non-technical users. The entire workflow is a single button with three states:

State	Appearance & Behaviour
Idle	Grey button labelled “Phrase Loop Start”. Turns purple-tinted during playback to indicate readiness
Capturing	Button turns green with pulsing dot. Hovering turns it red — a clear signal that clicking will cancel. Audio auto-plays if paused. Tap again to set the end point
Looping	Button becomes a solid teal pill showing the loop range (e.g. 0:23 – 0:45). Teal region highlighted on scrubber. Tapping restarts from phrase start

- End point set before start point: they swap automatically
- A × stop button appears alongside the looping button to cancel the loop
- Loop persists on part switch — see Section 3.1

3.4 PDF Viewer & Toolbar

- All PDF pages rendered at once in a scrollable column
- Zoom controls with 10 preset levels (50% to 200%), scroll position preserved on zoom change
- Fullscreen mode via the native browser Fullscreen API; on iOS a “fake fullscreen” mode hides the header and player bar, expanding the PDF area naturally
- **Auto-scroll:** Scroll Map: a tap-to-record system that teaches the app exactly how the sheet music moves with the song, then plays it back automatically. Updated in v7.3 to a

four-state machine: **State 0 — Idle**: tap **Scroll Lyrics** to begin recording. **State 1 — Recording**: the PDF auto-scrolls at the configured speed; controls shown are Pause Scrolling, speed slider, **Cancel** ✕ (discard all), and **Save** ✓ (commit map). Tap Pause Scrolling to enter State 2. **State 2 — Paused**: reposition if needed, then tap **Resume Scrolling** to continue or **Save** ✓ to commit. **State 3 — Autoscroll On**: the saved map plays back automatically on every listen; tap the toggle to pause/resume. Tap **Clear Map** (two-tap confirm) to discard and return to State 0. Teal segment markers appear on the scrubber when a map exists. The map saves per song in localStorage

- Scroll speed (0.25× to 5×) — controls how fast the PDF auto-scrolls during recording. At 1× the PDF takes 60 seconds to traverse its full height; adjust before recording to suit the pace of the song. Playback replays at the same rate as recorded, so recording and playback are always consistent. Only shown during the Recording (State 1) and Paused (State 2) states — hidden in Idle and Autoscroll On. Saved per song
- In fullscreen: media controls (play/pause, skip ±10s, scrubber) appear at the bottom in a slim toolbar

PDF Brightness & Invert Pill

A floating control sits in the bottom-left corner of the PDF area, just above the toolbar. It is designed to be unobtrusive but immediately accessible:

- **Collapsed**: shows only a small sun (☀️) icon at reduced opacity. Tap to expand
- **Expanded**: slides open rightward revealing – 80% + controls. Five steps: 25%, 40%, 60%, 80%, 100%
- **Invert**: once expanded, tapping the sun icon again toggles PDF inversion. White pages go dark — useful for night practice. The icon turns teal when active. Invert and brightness stack: you can have an inverted page dimmed further
- **Collapse**: tapping anywhere outside the pill collapses it. Invert state persists when collapsed; the teal icon serves as a reminder
- **Per-theme persistence**: brightness level and invert state are saved separately for light and dark mode (pdfBrightness_dark, pdfBrightness_light, pdfInvert_dark, pdfInvert_light). Switching theme restores whichever settings you last used in that theme

The brightness filter is applied only to the PDF canvas scroll area — the toolbar, player bar, and all UI chrome remain at full brightness.

PDF Toolbar Layout

- Left: page count, Scroll Lyrics button (four states: Idle/Recording/Paused/Autoscroll On), scroll speed –/×/+ (States 1–2 only), Cancel ✕ and Save ✓ (States 1–2 only), Clear Map ✕ (two-tap confirm, State 3 only)
- Centre: slim media controls (standard fullscreen only) — hidden in fake-fullscreen mode; full player bar remains visible below the PDF area in fake-fs
- Right: zoom –/×/+, Fullscreen button

3.5 Light & Dark Mode

- Moon/sun toggle in the header switches between dark and light colour schemes
- Dark mode (default): near-black background with warm purple undertone, gold and purple accents
- Light mode: warm lavender-white background with deepened accents for contrast

- Theme preference is persisted to localStorage (a genuine preference, unlike session-only controls)
- **Relationship to brightness/invert:** the PDF brightness pill is independent of the theme toggle. Both serve different needs — theme changes the app UI; the brightness pill adjusts only the PDF content. Per-theme brightness settings mean a user’s dark-room setup is automatically restored each time they switch to dark mode

3.6 Feature Guide (Welcome Screen)

On first load (or when no song is selected), a welcome screen displays a 3×3 feature grid. The grid is organised into three deliberate rows:

Row	Cards & Purpose
Practice trio (top)	Phrase Loop, Slow Down Playback, Switch Part Mid-Song — the three features that work best together when drilling a difficult passage. A teal full-width banner above the row reads “These three work best together”. Each card has a teal left-border accent tying it visually to the banner
Lyrics controls (middle)	Page Zoom, Scroll Lyrics, Fullscreen — controls for reading the sheet music comfortably
Preferences (bottom)	Your Default Part, Settings Saved Automatically, Light & Dark Mode — setup and personalisation

Slim section labels (“Lyrics controls”, “Preferences”) span the full grid width to orient users without adding a fourth card.

First-Load Onboarding (v7.6)

On first visit (no song selected), the song selector gains a pulsing gold border to draw the user’s eye immediately. A bouncing arrow appears below the selector as a further nudge. Once the user picks a song, the pulsing animation stops and a static gold border appears on both the song and part selectors, signalling that both need to be set before practice can begin. The onboarding state is managed in JavaScript and cleared once both selectors have values.

Welcome Hero (v7.7)

The empty state (no song loaded) now shows a full welcome hero above the feature cards: an animated mic logo, the Choir Geek title in the brand font, and a short tagline. This branding panel is visible on every load before a song is selected. Once a song is chosen and all feature cards have been dismissed, the hero remains as the sole content in the empty state, giving the app a polished starting screen rather than a blank canvas.

3.7 Settings Panel (v7.9 / v8.2)

A gear icon () sits in the header icon row on all device sizes. Tapping it slides down a settings panel immediately below the header. The panel animates open and closed using a CSS grid *grid-template-rows: 0fr → 1fr* transition (with *overflow: hidden* on the container to collapse padding to true zero). The panel closes on outside click or Escape key.

Panel contents (left to right): **Theme** toggle (light/dark), **Reload** (refresh the song list), **Help** (opens the feature guide), and the **Autoplay** toggle (session-only). The panel content is centred on all screen sizes. In fake-fullscreen mode the header and settings panel are both hidden; the player bar and iPhone mini-bar remain visible.

3.8 Settings Persistence

Setting	Scope & Notes
volume	Global — last set level (0–1)
theme	Global — dark or light
speedIdx	Session-only — resets to 1× on every new song
defaultPartLabel	Global — preferred part, auto-selected on every song
zoomMult	Per song — PDF zoom level
scrollMap / scrollSpeed	Per song — recorded scroll map segments and speed multiplier
pdfBrightness_dark / _light	Global per theme — PDF brightness level
pdfInvert_dark / _light	Global per theme — PDF invert state

autoplay is session-only and always resets to off. The scroll map persists per song. See Appendix for rationale.

3.9 Keyboard Shortcuts

Key	Action
Space	Play / Pause
← Arrow	Skip back 10 seconds
→ Arrow	Skip forward 10 seconds
P	Tap Phrase Loop button (start / stop capture)
Escape	Cancel active phrase loop
—	No longer assigned

3.10 Touch Optimisation

- All interactive elements meet 48px minimum touch target
- Play button is 56px for easy central tapping
- Progress bar thumb is 22px with a glow ring; bar track height 28px for a generous hit area

- :active states on all buttons provide immediate tap feedback
- Pinch-to-zoom enabled — user-scalable=no removed in v6.0 (accessibility requirement)

3.11 Responsive Device Layouts

Choir Geek has a full responsive design pass (v7.0–v8.2) that tailors the layout to each device class. The four key layouts are described below.

Desktop and Tablet Landscape ($\geq 1025\text{px}$)

Full layout. All controls visible in the header and player bar; header content is centred. A gear icon opens the settings panel (Theme, Reload, Help, Autoplay). The PDF toolbar shows at desktop sizing: Scroll Lyrics button with full state labels, scroll speed controls, zoom, and fullscreen. Tablet landscape (769px–1024px) uses the same toolbar sizing as desktop and also benefits from centred header content via an iPad-only breakpoint (min-width: 1024px, pointer: coarse).

Phone Portrait ($\leq 600\text{px}$, portrait, touch)

The header switches from flexbox to a 2-row CSS Grid. The CG logo spans both rows on the left. Row 1 holds the song selector and the icon buttons (gear/settings, theme, reset, help). Row 2 holds the part selector. The autoplay toggle moved to the settings panel in v7.9 — tap the gear icon to access it. This gives each control adequate space and touch target size without scrolling or overflow.

- Fullscreen button hidden in the PDF toolbar (not needed in portrait)
- When the Scroll Lyrics button is in the recording or paused state, the Zoom button label is hidden and only the icon and percentage remain — this saves horizontal space so the toolbar fits cleanly in one row
- The welcome screen feature grid collapses to a single column to prevent horizontal overflow

Phone Landscape Mini-Bar (max-height 430px, landscape, touch)

When an iPhone or Android phone is rotated to landscape, the browser chrome (address bar, tab bar) consumes so much vertical space that the full header and player bar would leave almost no room for the PDF. Choir Geek detects this via a CSS media query (max-height: 430px + orientation: landscape + pointer: coarse) and replaces both with a slim 52px fixed bar at the bottom of the screen. Once a track is selected the header disappears entirely, maximising the PDF area.

The mini-bar contains all the rehearsal-critical controls a singer needs mid-song, designed with the older demographic in mind — all buttons are 38px tall (matching the PDF toolbar buttons in visual weight) with clearly labelled text:

- **Loop** — full phrase looper with identical three-state behaviour to the desktop version. Idle: bordered button labelled “Loop”. First tap: button turns teal with a pulsing dot and label changes to “Loop End” — a clear prompt to tap again to mark the end of the phrase. Second tap: button becomes solid teal “Loop” and a red X cancel button appears. Tapping the solid Loop button restarts from the phrase start
- **Start** — skip-to-beginning icon button; resets track and PDF scroll to position zero
- Skip back 10s, Play/Pause, Skip forward 10s — standard transport controls
- **Slower** — simplified speed control optimised for one-handed mobile use. Tapping “Slower” immediately steps to –10% and replaces the button with a teal percentage pill (e.g. “–10%”) and a X cancel button. Each subsequent tap of the percentage pill steps

slower further (–18%, –25%, –30%, –35%). Tapping × resets to 1× and restores the “Slower” button. No separate + button — cancel is always faster than nudging back up

- Current time, scrubber progress bar, total duration — the scrubber fills the remaining width between the fixed controls

4. Home Assistant Integration (Optional)

The app no longer requires Home Assistant. If you wish to continue using HA as a front-end, the configuration from v1 remains valid: serve the Choir Geek folder from a local HTTP server on port 8124 and embed index.html in a Webpage card. Note: as of v6.0 the app uses native ES modules and requires serving over HTTP — it cannot be opened as a local file:// URL.

configuration.yaml entry	shell_command: generate_choir_tracks: "python3 /config/www/choir/generate_tracks.py" start_choir_server: bash /config/www/choir/start_server.sh
Lovelace card	type: iframe url: http://192.168.1.27:8124/choir-practice.html aspect_ratio: "100%"
Adding new songs	1. Copy song folder to /config/www/choir/ 2. Run shell_command.generate_choir_tracks 3. Press Refresh in the app

5. Design & Branding

5.1 Identity

Name	Choir Geek
Logo	Inline SVG: gold music note with purple sparkle and teal accent dot
Primary font	Playfair Display (display / brand)
Body font	DM Sans
Background (dark)	#0f0e11 — near-black with warm purple undertone
Background (light)	#f4f2f8 — warm lavender-white
Gold accent	#c8a96e
Purple accent	#9b6fd4
Teal (loop/active)	#4fc4a0
Muted text	#7d7892 (dark) / #71669a (light)

5.2 Layout Zones

- Header — branding, song/part selectors, light/dark mode toggle, refresh, and gear icon. Settings panel (below header) — slides open on gear tap; contains Theme toggle, Reload, Help, and Autoplay toggle. Responsive: at $\leq 820\text{px}$ brand name shrinks to “CG”; at $\leq 600\text{px}$ portrait the header switches to a 2-row CSS Grid layout with the CG logo spanning both rows, the song selector and icon buttons (gear/settings/theme/reset/help) on row 1, and the part selector on row 2. In phone landscape (mini-bar mode) the header is hidden once a track is loaded — see Section 3.11.
- Main area — PDF viewer fills all available vertical space; brightness pill floats in bottom-left corner
- PDF toolbar — auto-scroll, zoom, fullscreen (pinned to bottom of PDF area)
- Player bar — fixed at very bottom: phrase looper, transport, scrubber, volume/mute, slow down. Controls centred via 3-column CSS grid across 3 breakpoints

6. Market Analysis

6.1 Competitive Landscape

Product	Limitation
The Choir App	Combines PDF + MP3 but requires membership of a supported choir. Closed ecosystem
ChoirMate	Full choir management platform sold to administrators at ~\$25/month per group. Too complex for individual practice
forScore	PDF + audio sync. iOS/macOS only. Designed for instrumentalists, not singers
Paperless Music	MP3 + auto page turns. Aimed at instrumentalists. No harmony part management

6.2 The Gap Choir Geek Fills

- Bring your own files — no platform lock-in or membership required
- Multiple harmony parts per song, all linked to one PDF
- Phrase looper designed to be understood without instruction — three visual states, single button
- Loop persists when switching harmony parts — drill in Full Harmony then check your specific part
- PDF brightness and invert control — white lyric sheets in a dark room are a genuine pain point for this demographic, and no competitor addresses it
- Smooth 60fps auto-scroll paced to the audio
- Light and dark mode — accessible for users who prefer lighter interfaces or brighter environments
- Designed for non-technical users — every control is labelled, no icons without text
- Works on any device with a browser — no installation, no account

6.3 Target Market

- Contemporary rock choir members (Rock Choir alone has ~30,000 UK members)
- Community choir and amateur choir singers
- Anyone practising with MP3 files and PDF sheet music or lyric sheets
- Primarily middle-aged and older adults — simplicity and accessibility are paramount

7. Commercialisation Roadmap

Planning-stage thinking only. No decisions have been committed to.

Stage 1 — Validate the Problem

- Interview 10+ choir members about home practice habits — do not pitch, just listen
- 5–10 people willing to pay a specific price is meaningful validation
- Use existing choir contacts as the initial research pool

Stage 2 — Define the MVP

The minimum viable product needs only to do three things well: accept uploaded MP3s and a PDF, allow selection of a harmony part, and play the audio with lyrics visible. All other features are enhancements.

Stage 3 — Beta

- Recruit 20–30 beta users from existing choir contacts
- The demo site will already have all of the rock choir current setlist loaded. We will tackle the actual uploading of files and such later down the line once we prove the need for tis product.
- Free access in exchange for feedback
- Non-technical users will surface friction that developers miss
- Iterate on UX before public launch

Stage 4 — Infrastructure

- User accounts and authentication
- Cloud file storage (AWS S3 or Cloudflare R2)
- A backend to handle accounts and file management
- A domain — e.g. choirgeek.app (availability to be confirmed)

The current HTML/CSS/JS front-end is a solid foundation. The jump to a product is an infrastructure challenge, not a features problem.

Stage 5 — Monetisation Model

Tier	Details
------	---------

Free tier	Up to 5 songs, basic playback. Allows users to experience value before committing
Personal plan	~£3–5/month. Unlimited songs, all features, settings sync across devices
Choir plan	~£15–25/month. Conductor manages the library; all choir members get automatic access

The Choir plan — sold to administrators rather than individuals — offers the most revenue leverage. One sale covers an entire group.

Open Questions

- Technical path: build independently, find a co-founder, or hire?
- IP boundary: the app never hosts choir organisation content (users bring their own files) but the relationship with organisations like Rock Choir is worth understanding early
- Domain: choirgeek.app and choirgeek.co.uk should be checked and secured soon

8. Future Feature Roadmap

Features identified during the current build phase that are candidates for a future version. None are committed; all require validation against user needs before building.

8.1 Playlist / Practice Queue

Currently, users select one song at a time. A playlist feature would allow:

- A practice session queue — e.g. “tonight’s rehearsal set” in running order
- Automatic advance to the next song when a track ends (with autoplay on)
- Per-session playlist saved locally; no account required for personal use
- In a Choir plan context, a conductor could publish a playlist to all members — everyone practises the same songs in the same order before rehearsal

Technical path: extend tracks.json with an optional playlists array. The front-end already has all the playback machinery needed.

8.2 Practice Schedule Viewer

Rock Choir and similar groups often distribute a rehearsal schedule as a PDF — which songs are being rehearsed which week, new songs coming up, performance dates. At present, members keep this separately from the practice tool.

- A second PDF slot per song (or a global schedule PDF) loaded alongside the lyrics
- A toggle in the header to switch between “Lyrics” and “Schedule” view
- Could also surface as a separate tab or panel rather than replacing the lyrics view
- The schedule PDF could be a choir-wide shared file in a Choir plan, automatically updated by the conductor

Low implementation effort given the PDF viewer is already built. High perceived value for the target demographic, who currently juggle the app and a separate PDF or printed sheet.

8.3 Other Candidates

- **Favourite songs:** a star on song entries to mark songs currently being practised, separate from the default part star
- **Recently played:** surface the last 3–5 songs at the top of the selector for quick access
- **Notes per song:** a small free-text field per song for personal practice notes (“watch the bridge”, “entry on beat 3”)
- **Loop presets:** save named loop points per song (“Verse 1”, “Chorus”) to jump back to quickly without resetting manually

9. Accessibility & Final Polish

Choir Geek’s primary users are middle-aged and older adults — a demographic that is often underserved by software that assumes technical fluency and young eyes. The following considerations are grounded in that reality, not theoretical accessibility compliance.

9.1 Already Addressed

- 48px minimum touch targets on all interactive elements (56px for the play button)
- All interactive elements carry aria-label attributes — fully screen reader accessible
- High-contrast gold/teal active states on dark and light backgrounds
- PDF brightness and invert — directly addresses the bright-screen-in-dark-room problem common at evening choir sessions
- Stacked AUTO/PLAY label at narrow widths rather than hiding the toggle
- Three visual states on the phrase looper eliminate the need to read instructions
- Keyboard :focus-visible indicator — clear 2px accent outline for keyboard-only navigation (no effect on mouse use)
- Pinch-to-zoom re-enabled (user-scalable=no removed) — users with visual impairments can now zoom the interface
- WCAG AA contrast compliance for muted text — both dark (#7d7892) and light (#71669a) theme values now exceed 4.5:1

9.2 Recommended Next Steps

Screen Wake Lock

The single most impactful polish item for this demographic. iOS and Android will dim and lock the screen after 1–2 minutes of inactivity — while the user is standing at a music stand, singing, with their hands occupied. The **Wake Lock API** (supported in Chrome, Safari 16.4+, Edge) prevents this when audio is playing. Implementation is three lines of JavaScript. The lock is released automatically when audio pauses.

Respect prefers-reduced-motion

The app uses several CSS transitions and animations (the phrase loop pulse, the progress bar gradient, the brightness pill slide). Adding a @media (prefers-reduced-motion: reduce) override to disable non-essential animations respects the system accessibility setting for users who have motion sensitivity — a condition more common with age.

Offline / Service Worker

A simple service worker caching the HTML file, PDF.js library, and fonts would allow the app to load without an internet connection. For a choir member practising at home this is probably fine — but for someone in a rehearsal room, village hall, or touring with their choir, connectivity cannot be assumed. The MP3 and PDF files are local by design; only the app shell and CDN assets need caching.

Font Size Scaling

The UI text (selectors, player labels, button text) is already reasonable at 0.8–0.92rem, but a single “Large text” toggle in the header could scale the UI via a root font-size increase — beneficial for users who struggle with small text on a phone or who haven’t set their device’s accessibility text size. This is distinct from PDF zoom, which already handles lyric readability.

Error Recovery

Currently, if an MP3 file is missing or fails to load, the player silently stays in its inactive state. Adding an onerror handler on the audio element that surfaces a plain-language message (“Couldn’t load this track — check the file is in the right folder”) would prevent confusion for non-technical users who may not know to check the browser console.

Swipe to Skip

A horizontal swipe gesture on the PDF area (left/right) to skip –10s / +10s would allow eyes-up navigation on a phone without reaching for the player bar. Needs careful implementation to not conflict with the existing drag-to-scroll behaviour — a short fast swipe (velocity threshold) rather than a slow drag.

10. Deliverables

File	Purpose
index.html	Thin HTML shell linking CSS modules and JS modules. Served by Caddy at https://choirgeek.duckdns.org . Also requires styles/ and src/ folders — see Section 2.1.
generate_tracks.py	Track manifest generator. Run after adding new songs to regenerate tracks.json
Choir_Files_Renamer.py	Batch file renamer. Renames MP3s to the required Song - Part.mp3 convention
This document	Full project documentation: build, features, market analysis, roadmap

Appendix — Technical Notes

Why PDF.js instead of a native iframe?

Chrome blocks PDF rendering inside iframes when the parent page is served from a different origin or a non-standard port. PDF.js renders PDFs to HTML canvas elements, which are not subject to this restriction.

Why is auto-scroll driven by requestAnimationFrame?

The browser's timeupdate event fires only ~4 times per second, causing visible jumps in scroll position. A requestAnimationFrame loop at 60fps reads audio.currentTime directly each frame, producing smooth movement.

Why does the phrase looper auto-play when you set the start point?

If audio is paused when the user taps "Phrase Loop Start", the app automatically begins playback. This removes a step from the most common workflow: find the passage, tap start, let it play, tap end.

Why does the loop persist when switching harmony parts?

All harmony parts of a song share the same timing — they are all recordings of the same piece at the same length. Loop timestamps are equally valid for any part. Clearing the loop on part switch would punish exactly the behaviour the app is designed to encourage: drilling a phrase in Full Harmony then switching to your specific part. Only changing to a different song clears the loop.

Why is autoplay session-only (and why does the scroll map persist)?

Autoplay causes audio to start without user input, which is disorienting for less technical users — especially when switching songs. It is correctly session-only. The scroll map, by contrast, represents significant deliberate effort: a user records the scroll behaviour once and expects it to be there every time they return to that song. Persisting it is the right behaviour. Auto-scroll in the old sense (a simple speed multiplier running on every play) was session-only because it could race ahead of the PDF if the user had scrolled manually. The scroll map replaces this with an explicit, user-controlled recording that starts only when the map exists and the audio is playing.

Why no speed-up option?

Speeding up a vocal harmony track distorts pitch and timing in ways that actively harm muscle memory for singing. Slowing down to learn a difficult phrase is a well-established practice technique; speeding up is not. The speed control therefore only offers reduction: 1× down to 0.65× (−35%). The WSOLA pitch-preserving engine means pitch stays correct at every speed — unlike the browser's native playbackRate, which shifts pitch along with tempo.

How does the PDF brightness pill work technically?

A CSS filter: brightness(x) is applied to the pdf-canvas-scroll element only — not the surrounding UI. When invert is active, filter: invert(1) brightness(x) stacks both effects. The collapsed/expanded animation uses a max-width transition on overflow:hidden, so the icon stays anchored while the controls slide out rightward. Both brightness level and invert state are stored in localStorage keyed by theme (pdfBrightness_dark, pdfBrightness_light, pdfInvert_dark, pdfInvert_light), so switching theme silently restores per-theme settings.

How does the light/dark theme work technically?

All colours are defined as CSS custom properties (`--bg`, `--surface`, `--accent` etc.). A `[data-theme="light"]` attribute on the `<html>` element overrides these variables. The entire UI repaints from a single attribute change. JavaScript reads CSS variables at runtime when building gradient fills, so volume and scrubber gradients update correctly on theme switch.

How does fullscreen work on iPhone?

iOS Safari does not support the browser Fullscreen API. The app detects this at startup and falls back to a “fake fullscreen” mode: hiding the header and player bar via `body.fake-fs-active` causes the PDF area to expand naturally to fill the screen. This avoids `position:fixed` overlays, which cause iOS to repaint the entire compositing layer on every animation frame — producing visible flashing.

Why does switching harmony part reset the play state?

When a new audio source is assigned via `audio.src` and `audio.load()`, the browser stops playback but does not fire a pause event. The part-change handler explicitly resets the play/pause button and scrubbers to 0:00 so the UI always reflects reality.

Responsive design approach

The app uses a cascade of CSS media queries, built up through a dedicated responsive pass (v7.0–v8.2). From widest to narrowest: $\geq 1025\text{px}$ (desktop — full layout, all controls visible, header centred, gear icon opens settings panel); $\leq 1024\text{px}$ tablet landscape — PDF toolbar restored to full desktop sizing; iPad-only breakpoint centres the header; $\leq 820\text{px}$ — header brand name shrinks to “CG”; $\leq 600\text{px}$ portrait — header switches to a 2-row CSS Grid (see Section 3.11), fullscreen button hidden in PDF toolbar, zoom label abbreviated when scroll is active or paused; phone landscape max-height 430px — full header and player bar replaced by a slim fixed mini-bar (see Section 3.11). The autoplay toggle is in the settings panel on all screen sizes.

Integration approaches that did not work (Home Assistant)

Approach	Outcome
panel_custom with module_url	Expects a JavaScript module. Throws “Unable to load custom panel” error
panel_iframe	Removed from recent versions of Home Assistant
Webpage card (direct)	Chrome blocks the iframe due to sandboxing rules when accessed via the HA origin
shell_command with inline &	HA YAML passes <code>&</code> as a literal argument. Resolved by moving to a <code>.sh</code> script file

How does pitch-preserving slow-down work (WSOLA)?

The browser’s native `playbackRate` slows audio by changing the sample playback rate, which lowers pitch along with tempo — useless for learning a vocal melody. Choir Geek uses a custom inline implementation of the WSOLA (Waveform Similarity Overlap-Add) algorithm,

following the SoundTouch open-source library design by Olli Parviainen. The audio file is decoded in full via the Web Audio API (`AudioContext.decodeAudioData`) and held in an `AudioBuffer` in memory. A `SoundTouchWSOLA` class processes the buffer in real time using three parameters: a 12ms overlap window, a 28ms seek window, and an 82ms sequence length. Each cycle, it finds the best-matching position in the input via normalised cross-correlation (quick-seeking, stride-2), crossfades the overlap region to eliminate clicks, outputs a fixed number of frames, and advances the read head by $\text{tempo} \times \text{sequence}$. This yields time-stretching without pitch shift. A `ScriptProcessorNode` pushes the WSOLA output to the audio graph. The engine switches between native playback (at 1×) and WSOLA automatically as the user adjusts speed, with buffer prefetch beginning as soon as a track is selected so the decoded `AudioBuffer` is ready before the first “Slow Down” tap.

How does the Scroll Map system work technically?

The scroll map is an array of segment objects: $[[\{t_0, p_0, t_1, p_1\}]]$ where t is audio time in seconds and p is scroll position as a fraction of the maximum scroll range (0–1). The system uses a four-state machine (updated v7.3): State 0 (Idle) — no map active; State 1 (Recording) — $\{t_0, p_0\}$ captured on tap, PDF auto-scrolls at $\text{SCROLL_BASE_RATE} \times \text{scrollSpeed}$ (default: 1/60 of full height per second at 1×), Cancel ✕ discards all, Save ✓ commits; State 2 (Paused) — scrolling halted, user may reposition, Resume or Save ✓ to commit; State 3 (Autoscroll On) — saved map plays back automatically. Playback interpolates linearly from p_0 to p_1 over the recorded duration — no additional speed multiplier applied, so recording and playback rates are always identical. The `requestAnimationFrame` loop runs at 60fps and exits cleanly when no segment is active. The Clear Map button requires two taps (confirm pattern) and resets track and PDF to position 0. Both the scroll map and speed setting are stored per song in `localStorage`.